자료구조 실습

11/04

CONTENTS

오늘 실습 순서

- 이론 복습 — 포인터, new / delete, 구조체 포인터(->)
- 0단계 웜업: new로 노드 3개를 만들고 next로 잇기
- 단일 연결 리스트 — 문제마다 그림으로 동작 확인 -> 코드 빈칸 채우기 -> 실행 결과 확인
  1) create_node — 노드 1개 만들기
  2) sl_push_front — 맨 앞에 넣기
  3) sl_find · sl_push_back — 순회
  4) sl_remove · sl_clear — 연결 끊고 delete
- 이중 연결 리스트
  5) create_dnode · dl_push_front · dl_push_back
  6) dl_remove
- 문제 7: 실험 — 맨 뒤 삽입 SList vs DList

빌드 / 실행

```
g++ -Wall -o practice practice.cpp && ./practice
g++ -Wall -g -fsanitize=address -o practice practice.cpp && ./practice   # 메모리 오류 검사
```

REFERENCE

포인터와 동적 할당 — new / delete

- 포인터 = 주소를 담는 변수. *p로 가리키는 곳의 값을 읽고 씀
- new 타입 : 힙에 공간을 만들고 그 주소를 반환. delete p : 반납

오늘 쓰는 문법

동작	코드	비고
노드 1개 만들기	Node *node = new Node;	크기는 컴파일러가 계산
반납	delete node;	new 1번 = delete 1번
가리키는 곳 없음	nullptr	역참조하면 오류
멤버 초기화	node->next = nullptr;	new Node는 초기화 X

new / delete 기본형

```
int *p = new int;
*p = 15;
cout << p;    // 주소
cout << *p;   // 15
delete p;
p = nullptr;  // 허상 포인터 방지
```

```
delete 후에도 p에는 옛 주소가 남아 있음 -> 그 주소를 다시 쓰면 허상 포인터 (이론 '포인터 버그')
배열은 new[] / delete[] 짝이지만, 오늘은 노드를 하나씩 만들므로 new / delete만 사용
```

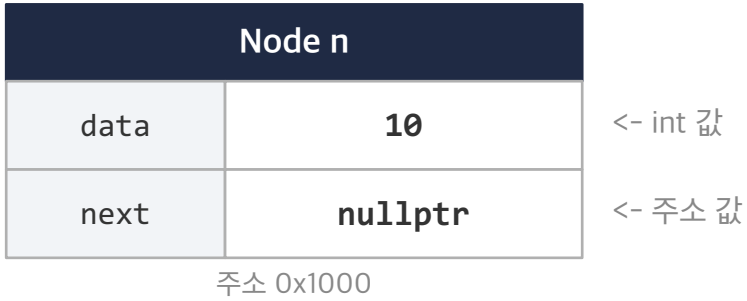
구조체 = 값 여러 개를 하나로 묶은 상자

노드 구조체 정의 (practice.cpp)

```
struct Node {
    int   data;   // 저장할 값
    Node *next;   // 다음 노드의 주소
};

Node n;           // 상자 1개 만들기
n.data = 10;      // 상자 n 안의 data 칸
n.next = nullptr; // 상자 n 안의 next 칸
```

메모리 그림



- Node n; 하면 data 칸과 next 칸이 붙어 있는 상자 1개가 생김
- 변수 이름 n 뒤에 . (점) 을 붙여 칸을 고름: n.data, n.next
- next는 Node * 타입 = '다른 Node 상자의 주소'를 담는 칸 -> 상자끼리 이어 붙일 수 있음

포인터로 상자에 접근할 때는 -> 를 쓴다

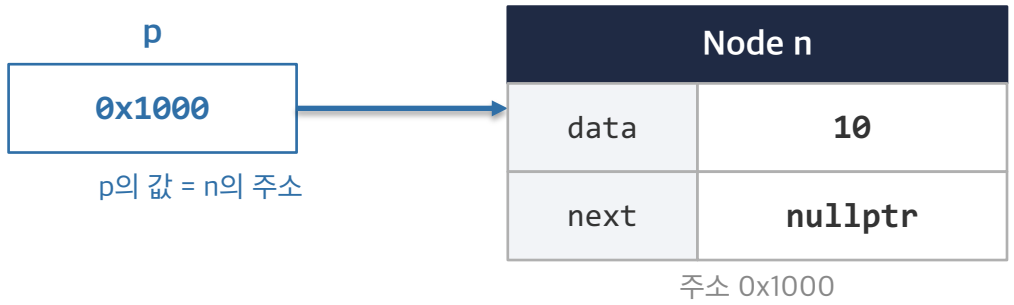
코드

```
Node n;
n.data = 10;

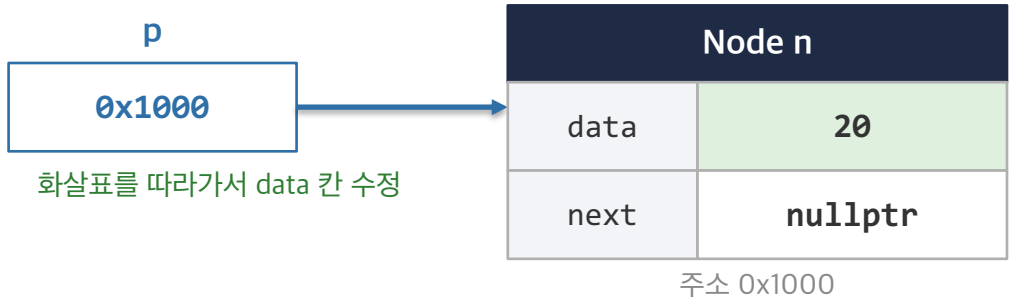
Node *p = &n;      // ① p에 n의 주소 저장
p->data = 20;      // ② p가 가리키는 상자의 data
(*p).data = 20;    //    ②와 같은 뜻
```

- 구조체 변수로 접근: n.data
- 포인터로 접근: p->data
- p->data = (*p).data
  *p = p가 가리키는 상자
  그 상자의 data 칸
- n.data도 20으로 바뀜
  (p와 n은 같은 상자)

① Node *p = &n;

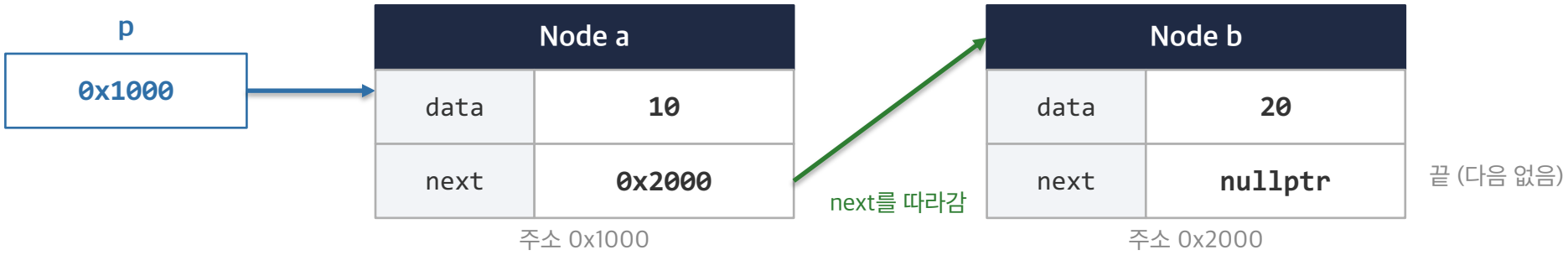


② p->data = 20;



오늘 코드의 list->head, node->next, cur->data 가 모두 이 형태 (list, node, cur 모두 포인터)

-> 를 이어 쓰면 next를 따라 다음 상자로 간다



식을 왼쪽부터 한 단계씩 읽기

식	뜻	값
p	a의 주소	0x1000
p->data	p가 가리키는 상자(a)의 data	10
p->next	a의 next 칸 = b의 주소	0x2000
p->next->data	a의 next가 가리키는 상자(b)의 data	20
p->next->next	b의 next 칸	nullptr

p->next->next->data 는 오류: nullptr이 가리키는 상자는 없음 -> 프로그램 비정상 종료
그래서 순회할 때는 항상 cur != nullptr 을 먼저 확인

new로 노드 3개를 만들고 next로 잇기

warmup( ) — 채울 것 없음, 실행해서 확인

```
Node *a = new Node;
Node *b = new Node;
Node *c = new Node;
a->data = 10; a->next = b;
b->data = 20; b->next = c;
c->data = 30; c->next = nullptr;

for (Node *cur = a; cur != nullptr; cur = cur->next)
    cout << cur->data << " ";

delete a; delete b; delete c;
```

그림



실행 결과 (주소는 실행마다 다름)

```
a: 주소 0x100a41860  data=10  next=0x100a41870
b: 주소 0x100a41870  data=20  next=0x100a41880
c: 주소 0x100a41880  data=30  next=0
a부터 next 따라가기: 10 20 30
delete 후 alloc_count = 0
```

- a의 next 값 = b의 주소. 주소를 따라가서 다음 노드에 도착 (nullptr은 cout에서 0으로 출력)
- 따로 new한 노드는 주소가 연속이라는 보장 없음 -> 배열처럼 인덱스로 못 감, next가 필요
- alloc_count = 아직 delete하지 않은 블록 수 (practice.cpp 위쪽에서 세어 줌). 다 지운 뒤 0이어야 누수 없음

REFERENCE

단일 연결 리스트 (SList) — head에서 출발해 next로 이동



리스트 구조체

```
struct SList {
    Node *head;   // 첫 노드 주소 (빈 리스트면 nullptr)
    int   size;   // 노드 개수
};
```

4주차 배열 리스트(UnsortedList)와 비교

	배열 리스트	단일 연결 리스트
저장 위치	연속된 칸, 크기 고정 (max_size)	노드마다 new, 흩어져 있음
i번째 원소	data[i] 바로 접근 · O(1)	head부터 i번 이동 · O(N)
맨 앞 삽입	모든 원소를 한 칸씩 밀기 · O(N)	포인터 2개 변경 · O(1)
크기	꽉 차면 더 못 넣음	메모리가 허용하는 한 계속

```
마지막 노드의 next = nullptr -> 순회는 cur == nullptr 이 되면 끝
노드를 따라가는 방법이 next 하나뿐 -> 뒤로(앞 노드로) 갈 수 없음
```

new로 노드를 만들고, 멤버 2개를 직접 초기화

코드 (노란 줄 = practice.cpp의 NEED_TO_SOLVE 자리)

```
Node *create_node(int data) {
    Node *node = nullptr;
    node = new Node;          // ①
    node->data = data;        // ②
    node->next = nullptr;     // ②
    return node;
}
```

create_node(7) 결과



- ① new Node
  힙에 Node 1개를 만들고 주소를 node에 저장
- ② data 저장, next = nullptr
  new Node는 멤버를 초기화하지 않음
  next에 쓰레기 주소가 남으면
  순회가 엉뚱한 메모리로 이동
- 반환값 = 새 노드의 주소
  push_front 등이 이 주소를 리스트에 연결

실행 결과

```
[PASS] 문제1 create_node(7): data=7 next=nullptr alloc=1
```

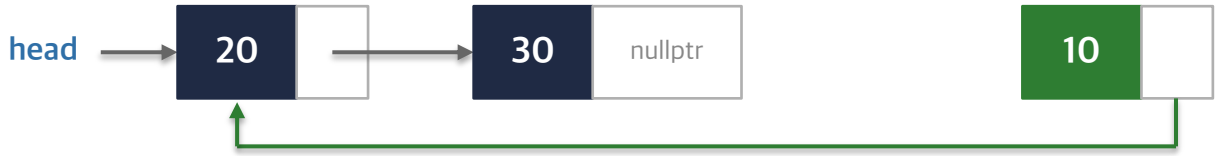
- alloc=1 : new 1회, 아직 delete 안 함 (체크포인트가 확인 후 delete)

새 노드가 기존 첫 노드를 먼저 붙잡고, 그다음 head를 옮긴다

시작: node = create_node(10)



① node->next = list->head;



node의 next가 20을 가리킴 (head와 같은 주소)

② list->head = node;



head -> [10] -> [20] -> [30] size = 3

```
순서를 바꿔 ② 먼저 하면: head가 node를 가리킨 뒤 node->next = head = node 자기 자신
-> [20] 이후 노드 주소를 잃음 (누수) + 순회하면 10만 무한 반복
```

포인터 2개만 바꾼다 — 리스트 길이와 무관하게 O(1)

코드 (노란 줄 = practice.cpp의 NEED_TO_SOLVE 자리)

```
void sl_push_front(SList *list, int data) {
    Node *node = create_node(data);
    if (node == nullptr) return;
    node->next = list->head;    // ①
    list->head = node;          // ②
    list->size++;
}
```

- ① 새 노드의 next = 지금의 첫 노드
  지금 첫 노드 주소는 list->head에 있음
- ② head = 새 노드
  ①을 먼저 해야 기존 노드 주소를 안 잃음
- 빈 리스트(head == nullptr)도 같은 코드
  node->next = nullptr, head = node

push_front 1, 2, 3 추적

```
push_front(1)  head -> [1]
push_front(2)  head -> [2] -> [1]
push_front(3)  head -> [3] -> [2] -> [1]
```

- 넣은 순서의 반대로 쌓임 (맨 앞에 계속 넣기 때문)

실행 결과

```
[PASS] 문제2 push_front 1,2,3:
  [3, 2, 1] size=3
```

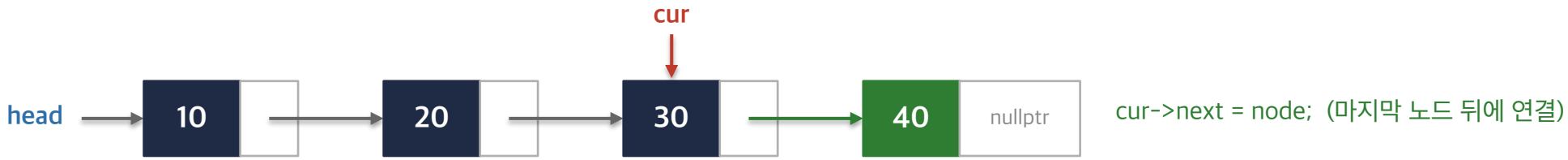
sl_find는 끝을 지날 때까지, sl_push_back은 마지막 노드에서 멈춘다

sl_find(99) : while (cur != nullptr)



- 모든 노드를 한 번씩 확인해야 함 -> 마지막 노드도 검사한 뒤 cur가 nullptr이 될 때 멈춤

sl_push_back(40) : while (cur->next != nullptr)



- 연결하려면 마지막 노드를 cur로 잡고 있어야 함 -> cur->next가 nullptr인 노드에서 멈춤
- 빈 리스트면 cur = head = nullptr -> cur->next 역참조 오류. 그래서 head == nullptr을 먼저 분기

끝까지 따라가야 하므로 둘 다 O(N)

코드 (노란 줄 = practice.cpp의 NEED_TO_SOLVE 자리)

```
Node *sl_find(SList *list, int value) {
    Node *cur = list->head;
    while (cur != nullptr) {
        if (cur->data == value) return cur;
        cur = cur->next;
    }
    return nullptr;
}

void sl_push_back(SList *list, int data) {
    Node *node = create_node(data);
    if (node == nullptr) return;
    if (list->head == nullptr) {
        list->head = node;
    } else {
        Node *cur = list->head;
        while (cur->next != nullptr) {
            cur = cur->next;
        }
        cur->next = node;
    }
    list->size++;
}
```

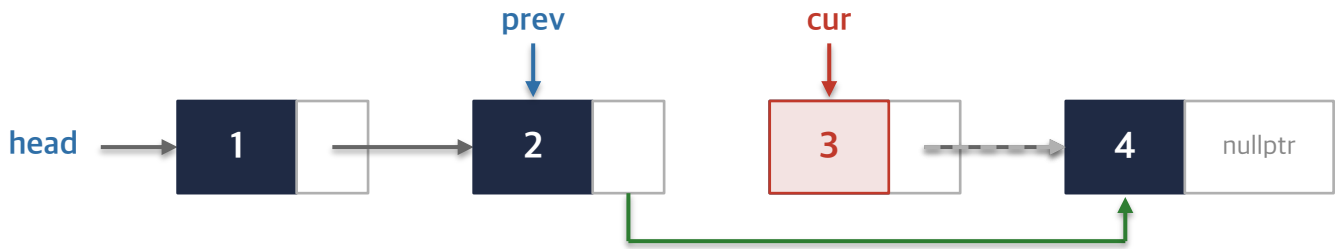
- 순회 기본형
  cur = head 에서 시작
  cur = cur->next 로 한 칸 이동
  cur == nullptr 이면 끝
- sl_find: 찾으면 그 노드 주소 반환
  끝까지 없으면 nullptr 반환
- sl_push_back
  빈 리스트면 head = 새 노드
  아니면 마지막 노드까지 이동 후
  cur->next = 새 노드

실행 결과

```
[PASS] 문제3 find 20 / 99:
  find(20)=20 find(99)=nullptr
[PASS] 문제3 back 2, front 1, back 3, back 4:
  [1, 2, 3, 4] size=4
```

앞 노드(prev)의 next를 cur 다음으로 바꾼 뒤 delete

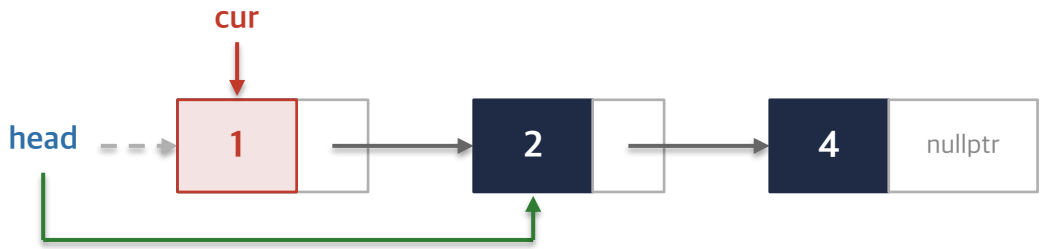
중간 노드 삭제: remove(3) — prev->next = cur->next;



그다음 delete cur

결과: head -> 1 -> 2 -> 4

첫 노드 삭제: remove(1) — prev == nullptr 이므로 list->head = cur->next;



고칠 next를 가진 앞 노드가 없음 -> head를 고침

- 단일 리스트는 뒤로 갈 수 없음 -> cur를 찾는 동안 바로 앞 노드를 prev에 기억
- 연결을 먼저 바꾸고 delete. delete를 먼저 하면 cur->next를 읽을 수 없음

연결 변경 -> delete 순서, alloc_count로 반납 확인

코드 (노란 줄 = practice.cpp의 NEED_TO_SOLVE 자리)

```
int sl_remove(SList *list, int value) {
    Node *prev = nullptr;
    Node *cur = list->head;
    while (cur != nullptr && cur->data != value) {
        prev = cur;
        cur = cur->next;
    }
    if (cur == nullptr) return 0;   // 못 찾음

    if (prev == nullptr) {          // 첫 노드
        list->head = cur->next;
    } else {
        prev->next = cur->next;
    }
    delete cur;
    list->size--;
    return 1;
}
```

```
void sl_clear(SList *list) {
    Node *cur = list->head;
    while (cur != nullptr) {
        Node *next = nullptr;
        next = cur->next;
        delete cur;
        cur = next;
    }
    list->head = nullptr;
    list->size = 0;
}
```

- remove: 반복이 끝나면 cur = 지울 노드,
  prev = 그 앞 노드 (첫 노드면 nullptr)
- clear: next 저장 -> delete -> 이동

실행 결과

```
[PASS] 문제4 [1..5]에서 3,1,5,9 삭제: [2, 4] size=2 ret=1110 alloc=2
[PASS] 문제4 sl_clear 후 남은 블록: 전 alloc=5 / 후 [] size=0 alloc=0
```

REFERENCE

이중 연결 리스트 (DList) — back 포인터와 tail 추가



단일 리스트에서 불편했던 점 -> 이중 리스트

	SList	DList
앞 노드 찾기	prev를 따로 기억	cur->back
맨 뒤 접근	head부터 O(N)	tail로 O(1)
노드 크기	data + 1포인터	data + 2포인터
고칠 포인터	적음	back까지 2배

구조체

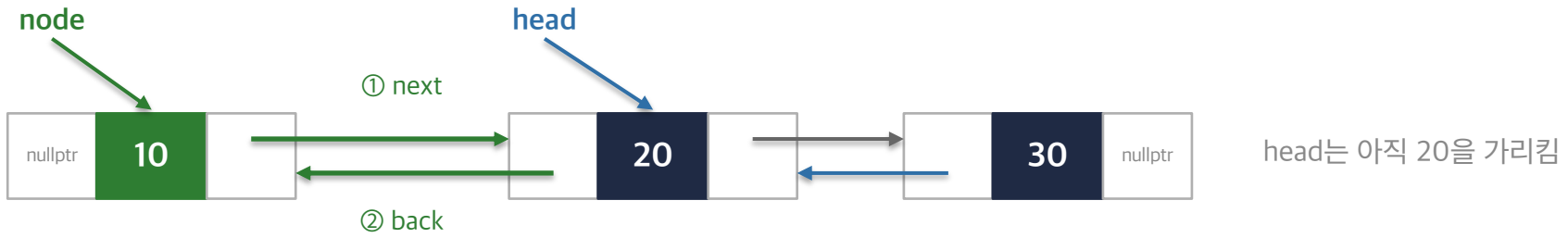
```
struct DNode {
    int data;
    DNode *back;   // 앞 노드
    DNode *next;   // 뒤 노드
};

struct DList {
    DNode *head;
    DNode *tail;   // 마지막 노드
    int size;
};
```

```
규칙: A <-> B 이면 A->next == B 이고 B->back == A  (항상 짝으로 맞춰야 함)
첫 노드의 back = nullptr, 마지막 노드의 next = nullptr
```

next 1개, back 1개, head 1개 — 고칠 포인터가 3개

① node->next = list->head; ② list->head->back = node;



head는 아직 20을 가리킴

③ list->head = node;



push_front / push_back 대칭

단계	dl_push_front	dl_push_back
①	node->next = head	node->back = tail
② 빈 리스트	tail = node	head = node
② 아니면	head->back = node	tail->next = node
③	head = node	tail = node

```
빈 리스트에 첫 노드를 넣으면
head와 tail이 모두 그 노드
(노드 1개 = 첫 노드이자 마지막 노드)
```

tail이 있으므로 맨 뒤 삽입도 O(1)

코드 (노란 줄 = practice.cpp의 NEED_TO_SOLVE 자리)

```
DNode *create_dnode(int data) {
    DNode *node = nullptr;
    node = new DNode;
    node->data = data;
    node->back = nullptr;
    node->next = nullptr;
    return node;
}

// dl_push_front
node->next = list->head;          // ①
if (list->head == nullptr) {      // ②
    list->tail = node;
} else {
    list->head->back = node;
}
list->head = node;                // ③
```

```
// dl_push_back (push_front와 대칭)
node->back = list->tail;          // ①
if (list->tail == nullptr) {      // ②
    list->head = node;
} else {
    list->tail->next = node;
}
list->tail = node;                // ③
```

- front: head 쪽, next로 연결, 기존 첫 노드의 back 수정
- back : tail 쪽, back으로 연결, 기존 끝 노드의 next 수정
- 빈 리스트에 넣으면 head와 tail 둘 다 새 노드

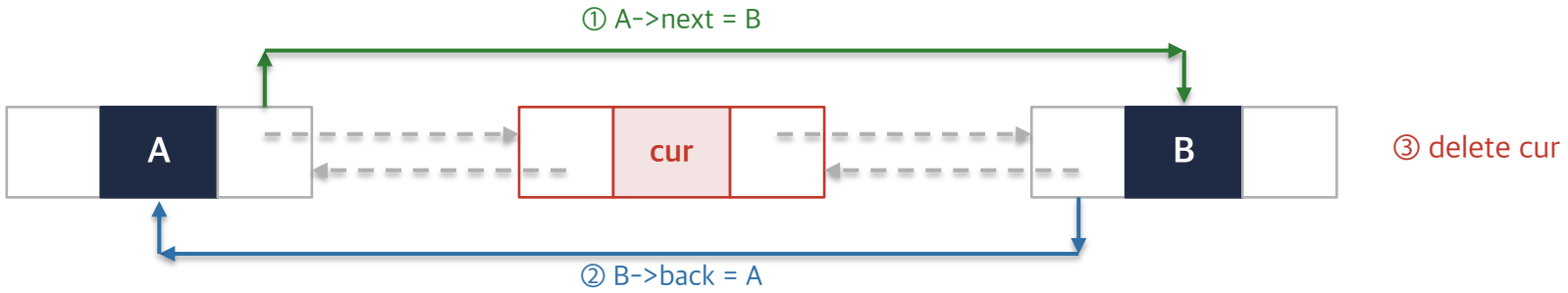
실행 결과

```
[PASS] 문제5 back 2, front 1, back 3, front 0:
  앞->뒤 [0, 1, 2, 3]
  뒤->앞 [3, 2, 1, 0] size=4
```

- 뒤->앞 = tail에서 back을 따라간 결과

앞 노드의 next, 뒤 노드의 back 두 군데를 고친다

중간 노드 삭제: cur->back->next = cur->next; cur->next->back = cur->back;



위치별로 바뀌는 것

cur 위치	① 앞쪽 (cur->back 기준)	② 뒤쪽 (cur->next 기준)
중간	A->next = B	B->back = A
첫 노드 (back == nullptr)	head = B	B->back = nullptr
마지막 (next == nullptr)	A->next = nullptr	tail = A
유일한 노드	head = nullptr	tail = nullptr

```
A = cur->back
B = cur->next
로 바로 접근
-> prev 불필요
```

앞쪽 1번, 뒤쪽 1번 고치고 delete

코드 (노란 줄 = practice.cpp의 NEED_TO_SOLVE 자리)

```
int dl_remove(DList *list, int value) {
    DNode *cur = list->head;
    while (cur != nullptr && cur->data != value) {
        cur = cur->next;
    }
    if (cur == nullptr) return 0;

    if (cur->back == nullptr) {       // ① 앞쪽
        list->head = cur->next;
    } else {
        cur->back->next = cur->next;
    }
    if (cur->next == nullptr) {       // ② 뒤쪽
        list->tail = cur->back;
    } else {
        cur->next->back = cur->back;
    }
    delete cur;                       // ③
    list->size--;
    return 1;
}
```

- cur->back = 앞 노드 -> prev 불필요
- ① 앞 노드가 없으면(첫 노드) head 수정
  있으면 앞 노드의 next 수정
- ② 뒤 노드가 없으면(마지막) tail 수정
  있으면 뒤 노드의 back 수정
- 노드가 1개면 ①, ② 모두 첫 분기
  -> head = tail = nullptr

실행 결과

```
[PASS] 문제6 [1..5]에서 3,1,5,9 삭제:
  앞->뒤 [2, 4] 뒤->앞 [4, 2]
  ret합=3 alloc=2
[PASS] 문제6 전부 삭제하면
  head/tail 모두 nullptr:
  ret합=3 head=nullptr tail=nullptr
  size=0 alloc=0
```

전부 통과하면 — 단일 리스트 들여다보기 출력

./practice 실행 시 체크포인트 다음에 출력됨

```
=== 단일 연결 리스트 들여다보기 (push_back 10, 20, 30) ===
  head = 0x100a41860
  [0x100a41860] data=10  next=0x100a41870
  [0x100a41870] data=20  next=0x100a41880
  [0x100a41880] data=30  next=0x0
```

- head 값 = 첫 노드 주소, 각 노드의 next 값 = 다음 줄 노드의 주소
- 마지막 노드 next=0x0 (nullptr)
- 통과 못한 항목이 있으면 dump와 실험은 실행되지 않음

FAIL 메시지 읽기

메시지	의심할 곳
비정상 종료 (nullptr 역참조 / 이중 delete)	nullptr인 cur에서 cur->next 접근, 같은 노드 delete 2번
2초 초과 (무한 루프)	cur = cur->next 빠짐, push_front 순서 바뀜 (자기 자신 가리킴)
alloc 값이 기대보다 큼	삭제 / clear 에서 delete 빠짐 (누수)

실험 — 원소 N개가 있을 때 push_back 1회: SList vs DList

측정 결과 (1회 평균, 실행 환경마다 값은 다름)

```
       N |    SList |   DList
---------------------------------
    4000 |   3.31us |  0.009us
    8000 |   9.45us |  0.007us
   16000 |  13.73us |  0.007us
   32000 |  26.93us |  0.007us
   64000 |  47.26us |  0.007us
```

```
SList : N이 16000 -> 32000 -> 64000 (2배씩)이면 13.73 -> 26.93 -> 47.26us (약 1.8~2배씩) -> O(N)
DList : N이 커져도 0.007~0.009us 그대로 -> O(1)
```

- 절대 시간이 아니라 'N이 2배가 될 때 몇 배가 되는가'를 볼 것
- N이 작을 때(4000 -> 8000)는 캐시 등 영향으로 비율이 고르지 않을 수 있음
- 64000개일 때 맨 뒤 삽입 1번에 약 6,700배 차이
- 실험 후 alloc_count = 0 -> sl_clear, dl_clear가 모두 반납

문제 7 · 해석

차이는 '마지막 노드를 아는가'에서 나온다

- SList: 마지막 노드 주소를 저장하지 않음 -> 매번 head부터 N번 따라가서 찾음
- DList: tail에 마지막 노드 주소가 있음 -> 바로 연결, 노드 수와 무관
- 대신 DList는 노드마다 back 포인터 1개(8바이트)를 더 씀

연산별 비교 (원소 N개)

연산	SList	DList	차이 이유
맨 앞 삽입	O(1)	O(1)	head로 바로 접근
맨 뒤 삽입	O(N)	O(1)	tail 유무
값 찾기	O(N)	O(N)	둘 다 순회
찾은 노드 삭제	prev 필요	back으로 바로	앞 노드 접근

배열 리스트(4주차)는 i번째 접근이 O(1), 연결 리스트는 앞/뒤 삽입·삭제가 O(1)
-> 어떤 연산을 자주 하는가에 따라 구조를 고른다

생각해보기

실행 마지막에 출력되는 질문 3개

1. sl_push_front의 ①, ② 순서를 바꾸면?

- head = node 를 먼저 하면 node->next = head = node -> 자기 자신을 가리킴 (문제 2 설명 참고)
- 기존 노드 주소를 잃음 (누수) + 순회가 끝나지 않음

2. push_front(Node *head, int data) 처럼 head를 값으로 받으면? (swap1과 비교)

```
void push_front(Node *head, int data) {
    Node *node = create_node(data);
    node->next = head;
    head = node;       // 사본만 바뀜
}
```

- head는 main의 head 값을 복사한 지역 변수
  -> swap1처럼 사본만 바뀌고 원본 그대로
- 해결: 원본의 주소를 넘김
  push_front(SList *list, ...) (오늘 코드, 주소 전달)
  push_front(Node *&head, ...) (참조 전달, swap2)

3. SList에 tail만 추가하면 맨 뒤 삭제도 O(1)이 되는가?

- O(1)이 되지 않음. 삭제 후 tail은 마지막 노드의 '앞 노드'가 되어야 함
- 단일 리스트는 앞 노드로 갈 수 없음 -> head부터 따라가야 함 -> O(N)
- DList는 tail->back 으로 바로 찾음 -> O(1)

SUMMARY

오늘 구현한 연산의 Big-O

연산	하는 일	Big-O
`create_node / create_dnode`	new 1회 + 필드 초기화	O(1)
`sl_push_front / dl_push_front`	포인터 2~3개 변경	O(1)
`sl_find`	head부터 순회	O(N)
`sl_push_back`	마지막 노드까지 이동 후 연결	O(N)
`dl_push_back`	tail 뒤에 바로 연결	O(1)
`sl_remove / dl_remove`	찾기 O(N) + 연결 변경 O(1)	O(N)
`sl_clear / dl_clear`	노드마다 delete	O(N)

```
new한 노드는 반드시 delete (alloc_count = 0 확인)
delete 전에 필요한 주소(next)를 먼저 저장 / 역참조 전에 nullptr 검사
이중 연결 리스트는 next와 back을 항상 짝으로 고친다
```

9주차 실습 끝

수고하셨습니다